Course: Individual Software Development Process' 2026

Software Requirement Specification

By **sonnyboy**

Chosen Irl Challenge: Project Aj. Milk : Lab Workflow & Request Management

Pannathon Nithiwatcharin 6710545695

Krittin Konsiang 6510545241

Paramee Saejia 6710545709

Piyapong Ausawarachan 6710545725

Phubet Ueananta 6710545814

Table of Contents

1. Target Users

- **Lab Members (Students):** Submit requests (e.g., experiment setup, equipment issues, consumables ordering) and track the status of their own requests.
- **Teaching Assistants (TAs):** Triage incoming requests, handle routine tasks, and update request status/progress.
- **Lecturers / Researchers:** Submit and track requests related to experiments, projects, or industry collaborations; view resource usage relevant to their work.
- **Lab Manager (Aj. Milk):** Oversee all requests across the lab, assign priority/ownership, manage equipment and consumables, and review overall lab activity.
- **External Industry Partner (indirect, non-system user):** Initiates or contributes to industry-collaboration requests, which are recorded and managed in the system by a Lecturer/Researcher on their behalf. External partners do not access the system directly in this iteration.

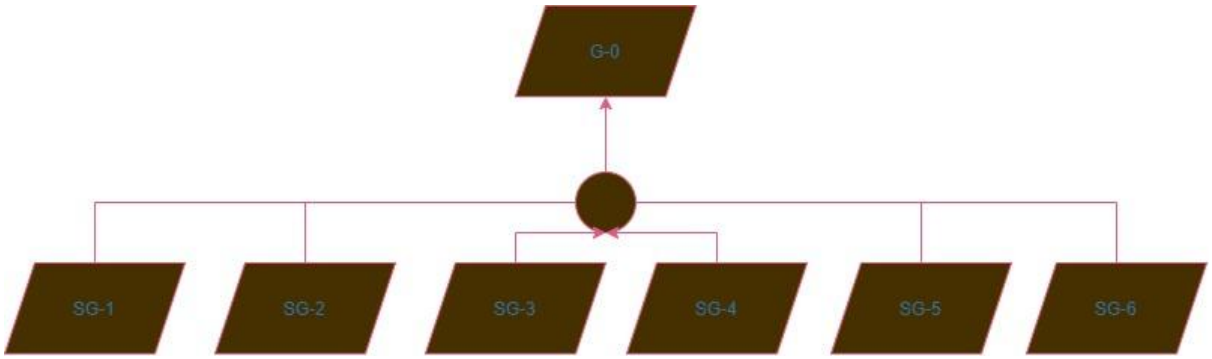
2. The System Goal

G-0: Make lab workflows and requests visible, trackable, and reliable.

The lab ("vase") currently handles day-to-day workflows — experiment setup requests, equipment maintenance, access permissions, consumables ordering, and small internal tickets — through email threads, chat messages, and ad-hoc spreadsheets. This causes important requests to get lost, priorities to be unclear, follow-up to be hard to track, and gives new lab members no central place to see "what's going on" or how to request support.

For this iteration, the system scope is limited to **request submission, viewing, status tracking, and basic triage/closure** of lab requests. Full inventory/equipment reservation systems, automated scheduling, and detailed analytics/reporting are explicitly out of scope for this iteration and will be addressed in later iterations.

3. Goal Refinement via KAOS:



Sub-Goal 1 (SG-1)

Description: Achieve: Provide a single, central place for lab members to submit and view requests.

Rationale: New lab members currently have no central place to see ongoing activity or request support.

Sub-Goal 2 (SG-2)

Description: Maintain: Complete and accurate request details (type, requester, description, priority, status).

Rationale: Incomplete or scattered request information across email/chat makes requests easy to misplace or misprioritize.

Sub-Goal 3 (SG-3)

Description: Achieve: Allow authorised staff (Lab Manager) to triage, assign, and update the status of requests.

Rationale: Requests currently have no clear owner or defined follow-up process.

Sub-Goal 4 (SG-4)

Description: Avoid: Requests being lost or left without follow-up.

Rationale: Email threads and ad-hoc spreadsheets currently cause important requests to be forgotten or delayed.

Sub-Goal 5 (SG-5)

Description: Maintain: A historical record of resolved/closed requests.

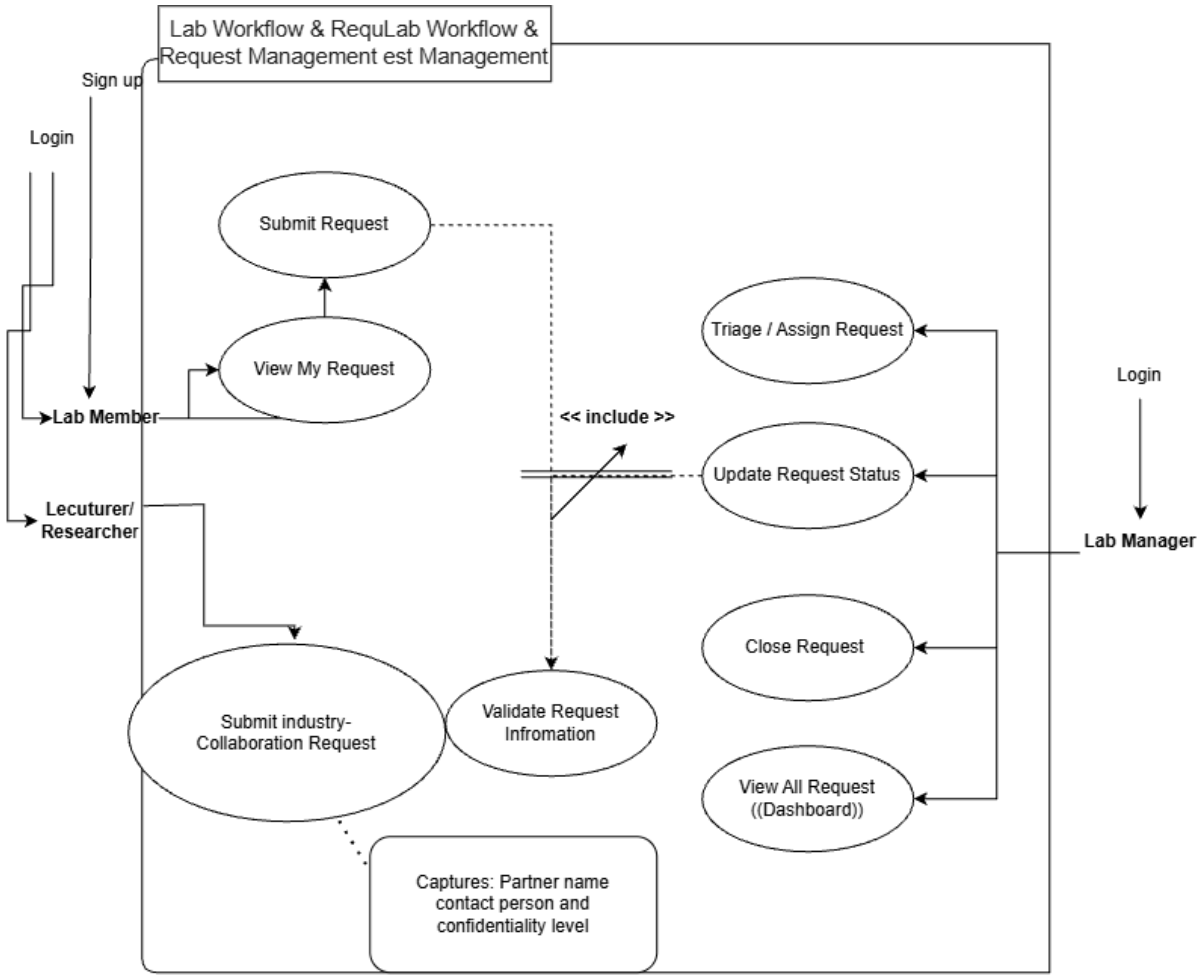
Rationale: It's currently difficult to see how lab resources and time are being used over time; retaining closed-request history supports this visibility and accountability.

Sub-Goal 6 (SG-6)

Description: Support industry-collaboration requests with a named external partner, contact person, and confidentiality level.

Rationale: Industry collaboration requests carry obligations (deliverables, confidentiality, external accountability) that generic lab tickets do not. Treating them the same causes information leakage risk and lost visibility for the Lab Manager who is accountable to the partner.

4. Use Case Diagram



5. Use Stories

User Story ID	Description	Acceptance Criteria
US-1	As a lab member, I want to submit a request (e.g., equipment issue, consumables order, script fix) so that my need is recorded and visible.	Request form requires type, description, and requester. A valid request is saved with status "New" and appears in the request list.
US-2	As a lab member, I want to view the status of my own requests so that I know if it's being worked on.	Lab members can see a list of their submitted requests with current status and last update time.
US-3	As a Lab Manager, I want to triage incoming requests so that priority and ownership are clear.	An authorised Lab Manager can set priority and assign an owner. The request list reflects the assignment.

US-4	As a Lab Manager, I want to update the status of a request (e.g., In Progress, Blocked, Done) so that everyone can track progress without asking in chat.	Status can only be changed by an authorised user. The requester's view reflects the updated status.
US-5	As a Lab Manager, I want to close a resolved request so that it no longer appears as open/pending.	Closed requests are removed from the active list but retained in history/records.
US-6	As a Lab Manager, I want to view all requests across the lab in one place so that I can see what's going on and how resources are being used.	Dashboard shows all requests with filters by status, type, and requester.
US-7	As a Lecturer/Researcher, I want to submit a request tagged as an Industry Collaboration with the partner company name, external contact, and confidentiality level, so that the Lab Manager can handle it appropriately.	When request type is "Industry Collaboration," the form requires **partner company name** and **external contact** before it can be saved. - Confidentiality level defaults to **"Restricted"** if not otherwise specified. - The request is rejected with a validation message if partner name or external contact is missing. - Once saved, the request is tagged and stored with its confidentiality level alongside the standard request fields.
US-8	As a Lab Manager, I want industry-collaboration requests to be visible only to the requester, assigned staff, and Lab Manager — not to all lab members — so that partner information stays confidential.	Requests marked **"Restricted"** do not appear in the general request dashboard/list for unauthorised users. - Only the original requester, the staff member(s) assigned to the request, and the Lab Manager can view a Restricted request's full details. - An unauthorised lab member attempting to access a Restricted request directly (e.g., via link) is denied and shown an appropriate access-restricted message. - Restricted requests are still included

		in the Lab Manager's full dashboard view (per US-6), but excluded from any general/shared view visible to all lab members.
US-09	As a Lab Manager, I want to change the user “role”, so that if any new Lecturer registers, I can give them the role.	- View all the users. - Can sort any user by time registered, email, role.

6. User Requirement Specification

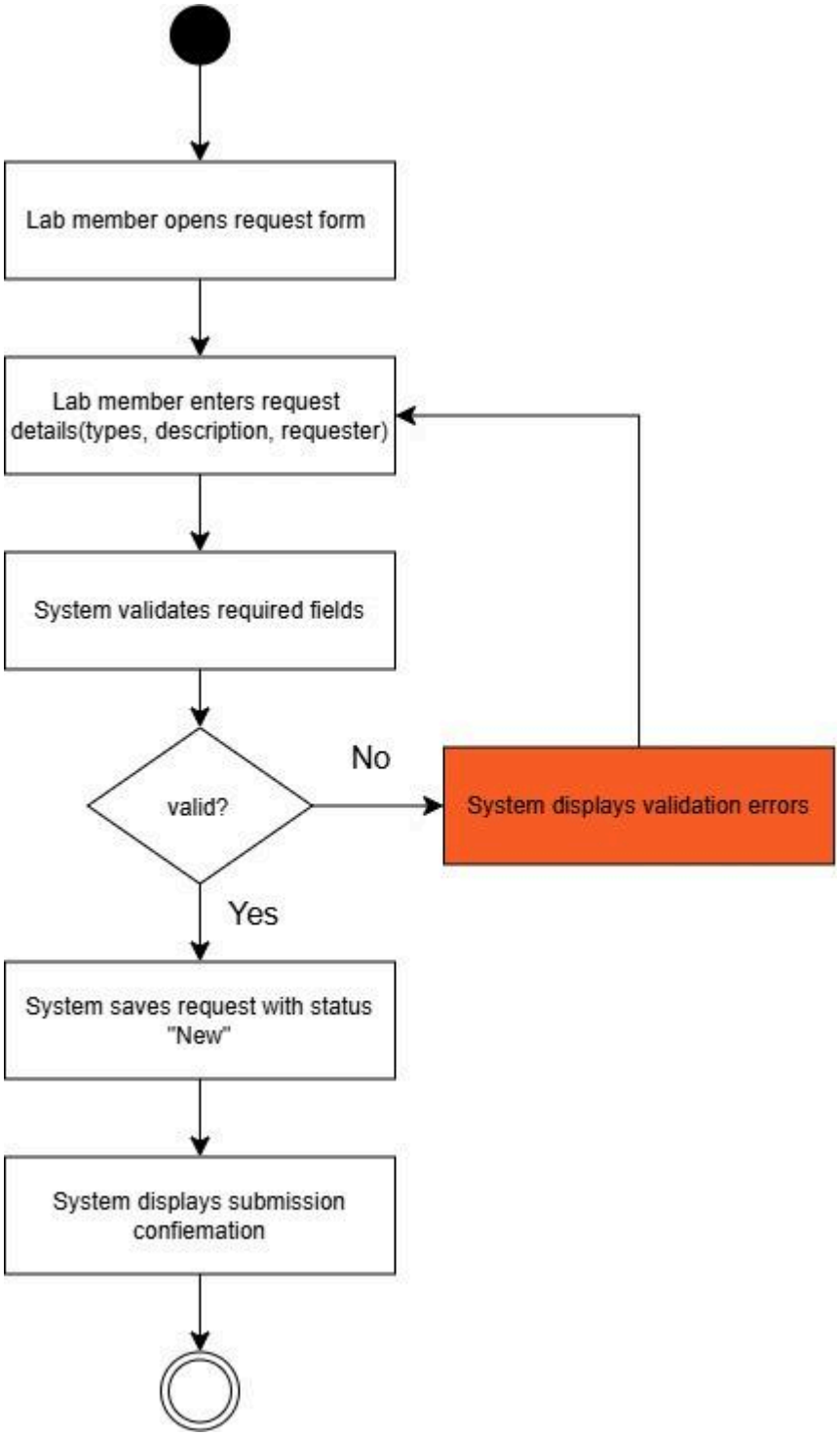
ID	User Requirement
URS-1	Lab members shall be able to submit a request with a type, description, and requester identity.
URS-2	Lab members shall be able to view the status and details of their own submitted requests.
URS-3	The system shall allow an authorised Lab Manager to set priority and assign ownership of a request.
URS-4	The system shall allow an authorised Lab Manager to update a request's status.
URS-5	The system shall allow an authorised Lab Manager to close a resolved request.
URS-6	The Lab Manager shall be able to view a dashboard of all requests, filterable by status, type, and requester.
URS-7	The system shall support an "Industry Collaboration" request type with additional fields: partner organisation, external contact name/email, and confidentiality level (Public / Internal / Restricted).
URS-8	The system shall restrict visibility of "Restricted" requests to the requester, assigned owner, and Lab Manager.

URS-9	Users shall be able to sign in using their Google account, and those signing in for the first time shall automatically be assigned the role of Lab Member.
URS-10	The Lab Manager or Lecturer shall be able to change the role of any user in the system.

7. Activity Diagram

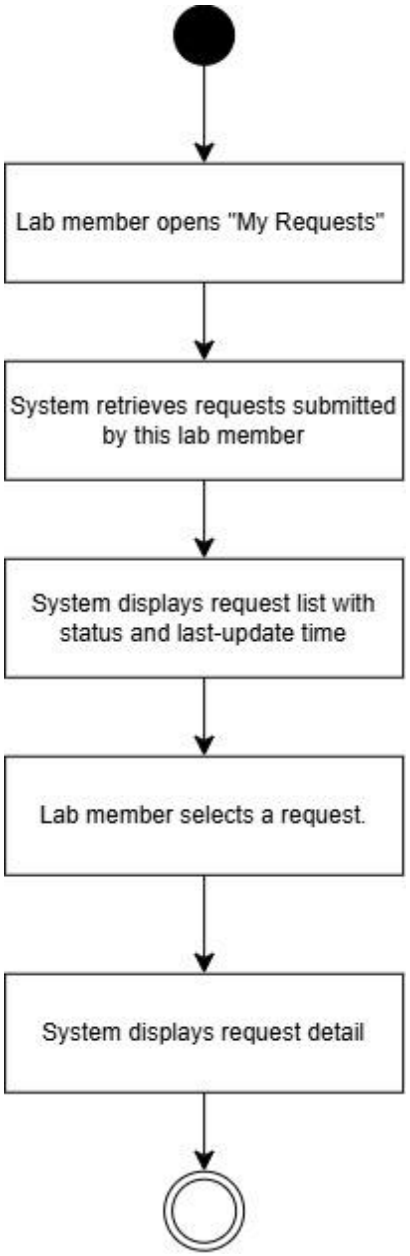
AD-1: Lab Member Submits a Request

This diagram traces the request-submission flow used to derive SRS1, SRS2, and SRS3

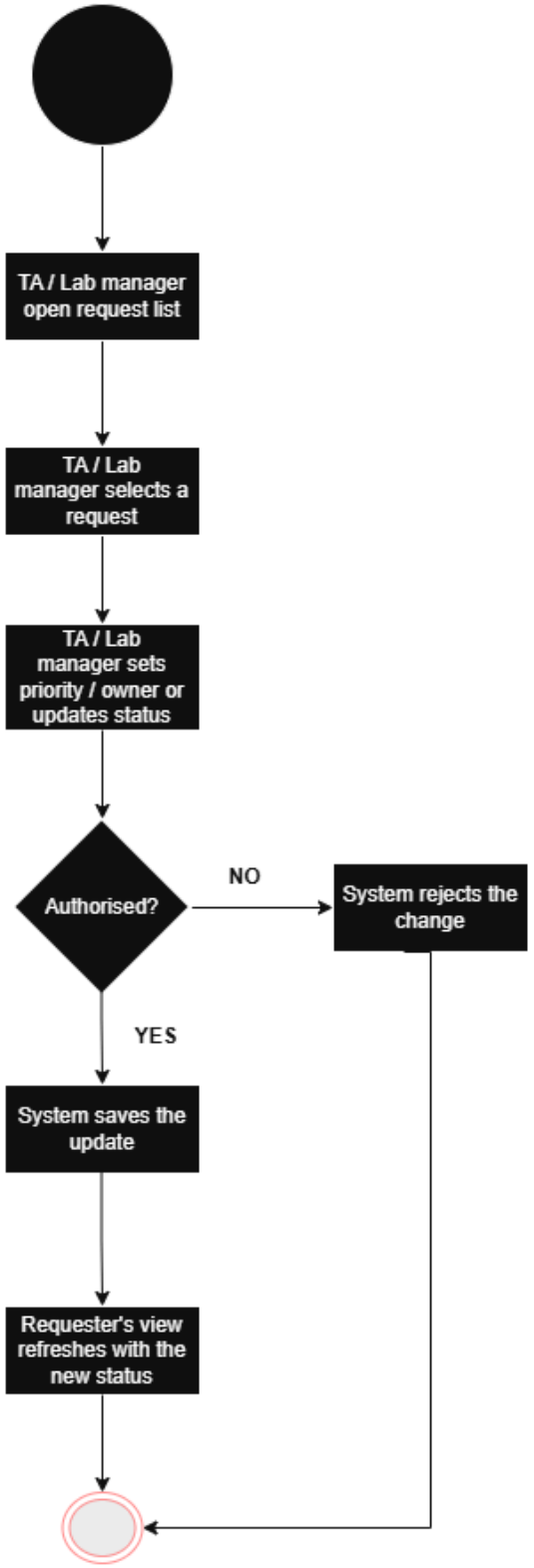


AD-2: Lab Member Views Their Request Status

This diagram traces the requester-facing status-viewing flow used to derive SRS-4 and SRS-5.

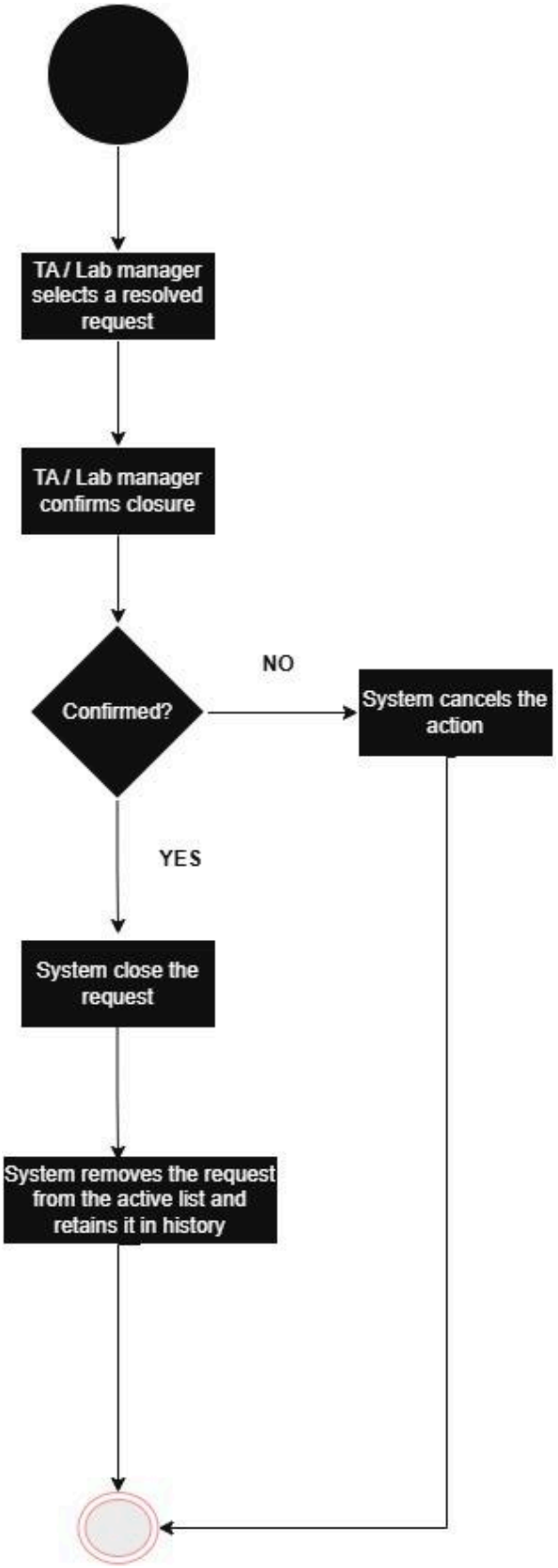


AD-3: Lab Manager Triages and Updates Request Status
This diagram captures the authorisation check and status-update flow used to derive SRS-6,SRS-7, SRS-8, and SRS-9.

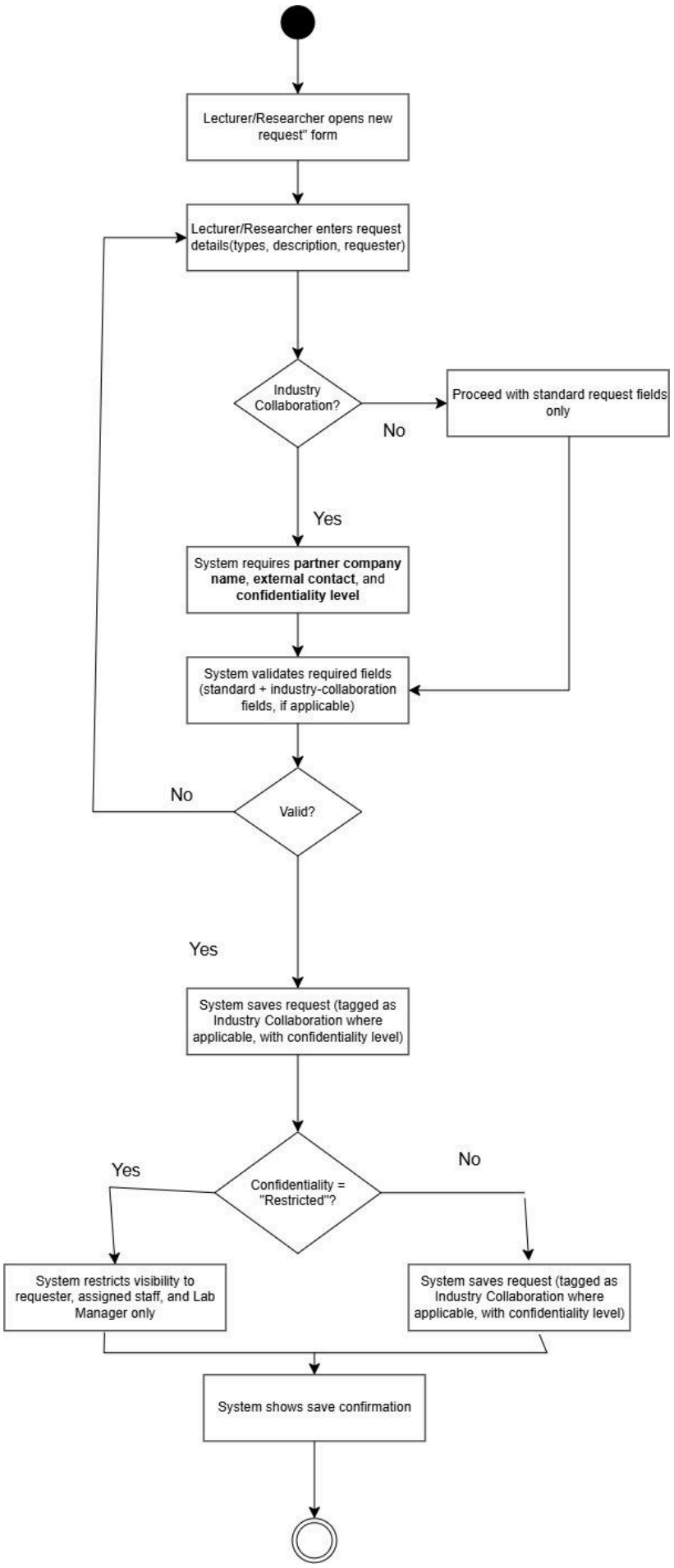


AD-4: Lab Manager Closes a Request

This diagram captures the confirmation and closure flow used to derive SRS-10 and SRS-11.



AD-5: Lecturer/Researcher Submits an Industry Collaboration Request
This diagram captures the additional field-requirement and visibility-restriction flow used to derive SRS-14 SRS-14 SRS-15 SRS-16 SRS-17


Lecturer/Researcher opens new request" form
Lecturer/Researcher enters request details(types, description, requester)
Industry Collaboration?
No
Proceed with standard request fields only
Yes
System requires partner company name, external contact, and confidentiality level
System validates required fields (standard + industry-collaboration fields, if applicable)
No
Valid?
Yes
System saves request (tagged as Industry Collaboration where applicable, with confidentiality level)
Yes
Confidentiality = "Restricted"?
No
System restricts visibility to requester, assigned staff, and Lab Manager only
System saves request (tagged as Industry Collaboration where applicable, with confidentiality level)
System shows save confirmation

8. System Requirement Specification

ID	System Requirement
SRS-1	The system shall provide a request submission form requiring request type, description, and requester identity before a request can be saved.
SRS-2	The system shall display a validation message and shall not save the request when required fields (type, description, requester) are missing or invalid.
SRS-3	The system shall assign a newly saved request a default status of "New" and record the submission timestamp.
SRS-4	The system shall allow lab members to view a list of their own submitted requests, including current status and last-updated timestamp.
SRS-5	The system shall restrict lab members from viewing or editing requests submitted by other lab members.
SRS-6	The system shall allow only an authorised Lab Manager to set priority and assign ownership of a request.
SRS-7	The system shall allow only an authorised or Lab Manager to change a request's status (e.g., New, In Progress, Blocked, Done).
SRS-8	The system shall reject any triage or status-change action attempted by an unauthorised user and display an authorisation error.
SRS-9	The system shall immediately update the requester's view to reflect any change in status, priority, or ownership.
SRS-10	The system shall allow only an authorised or Lab Manager to close a resolved request.
SRS-11	The system shall remove closed requests from the active/open request list while retaining them in a historical record accessible to authorised staff.
SRS-12	The system shall provide the Lab Manager with a dashboard view of all requests, filterable by status, type, and requester.
SRS-13	The system shall persist all submitted requests until explicitly closed, preventing requests from being silently lost.

SRS-14	The system shall provide "Industry Collaboration" as a selectable request type; when selected, partner organisation, external contact, and confidentiality level shall be required before the request can be saved
SRS-15	The system shall default the confidentiality level of Industry Collaboration requests to "Restricted."
SRS-16	The system shall hide requests marked "Restricted" from any user who is not the requester, the assigned owner, or the Lab Manager — including from the general dashboard (SRS-12)
SRS-17	The system shall log all access to "Restricted" requests (who viewed, when) for accountability
SRS-18	The system shall authenticate users via Google OAuth 2.0.
SRS-19	The system shall automatically create a new user record with role = "lab_member" and record the registration timestamp when an unrecognised Google account signs in for the first time.
SRS-20	The system shall allow only a Lab Manager or Lecturer to update the role of any user.

9. Software Architecture

Based on the SRS for the Lab Workflow & Request Management system (Iteration 1 scope: request submission, viewing, triage/status updates, closure, dashboard, and industry-collaboration requests), the recommended architecture is a Modular Monolithic Architecture using the Model-View-Controller (MVC) pattern. This is a single deployable application, internally organised into clearly separated layers, rather than a flat monolith or a distributed microservices system.

Rationale

The system serves a small, well-defined user group (Lab Members, Lecturers or Researchers, and one Lab Manager) within a single lab environment. A full microservices architecture would introduce unnecessary operational overhead — service discovery, inter-service networking, and multiple deployments — that the development team cannot realistically maintain for this scope. A flat, undifferentiated monolith, on the other hand, risks becoming difficult to maintain as role-based authorisation logic (SRS-5 through SRS-8, SRS-10, SRS-16), confidentiality enforcement (SRS-16, SRS-17), and request-lifecycle rules (SRS-3, SRS-7, SRS-9, SRS-11) grow more complex over future iterations (inventory reservation, scheduling, and analytics are

already flagged as later scope). The MVC modular approach gives the simplicity of a single deployment while cleanly separating presentation (View), request handling and authorisation (Controller), and business/data logic (Model), keeping the codebase maintainable and leaving the door open to extracting individual modules into microservices later if the system's user base or feature set grows significantly.

Component	Layer/Type	Description
Web / Mobile Client	Presentation (View)	Renders the public request list and submission form for Lab Members and Lecturers/Researchers; renders the triage/status-update interface for Lab Manager; renders the full dashboard for the Lab Manager. Displays confirmations, validation messages, and access-restricted notices.
Request Controller	Controller	Receives HTTP requests (submit, view my requests, triage, update status, close, view dashboard), performs role-based authorisation checks (SRS-5 through SRS-8, SRS-10), enforces confidentiality access rules (SRS-16), and coordinates calls to the Model layer before returning a response to the View.
Request Model / Business Logic	Model	Encapsulates request business rules: validates required fields (SRS-1, SRS-2), assigns default status 'New' and submission timestamp (SRS-3), enforces status transition logic (SRS-7), applies closure logic (SRS-10, SRS-11), validates industry-collaboration extra fields (SRS-14, SRS-15), and applies confidentiality-based visibility filtering (SRS-16).
Request Database	Persistence	Stores request records (type, description, requester, status, priority, owner, timestamps, industry-collaboration fields, confidentiality level, archived flag) and access logs (SRS-17). Serves

Component	Layer/Type	Description
		filtered queries for the requester view, Lab Manager triage list, and Lab Manager dashboard.
Auth / Authorisation Module	Cross-cutting	Verifies user identity and role (Lab Member, TA, Lecturer/Researcher, Lab Manager) before any write or restricted-read operation; used by the Controller to gate triage (SRS-6), status changes (SRS-7, SRS-8), closure (SRS-10), Restricted-request access (SRS-16), and role management (SRS-20).
Access Log Module	Cross-cutting	Records who accessed a Restricted request and when (SRS-17); called by the Controller after every successful read of a Restricted request.

10. Data Storage Technology

The recommended data storage technology is a Relational Database Management System (RDBMS), such as PostgreSQL or MySQL, used as the primary data store for this iteration, supplemented by a lightweight structured log table for access auditing.

Rationale

All core data described in the SRS is structured and fits a fixed, well-defined schema: every request has a type, description, requester identity, priority, status, owner, submission timestamp, last-updated timestamp, and optional industry-collaboration fields (partner organisation, external contact, confidentiality level). There are no nested, variable-shape, or rapidly evolving data structures in this iteration that would justify a document-oriented or key-value NoSQL store. An RDBMS additionally enforces referential integrity and data constraints (required fields, valid status values), and supports transactional consistency, which directly satisfies SRS-1, SRS-2, SRS-13 (requests must not be silently lost), and SRS-9 (status changes are immediately and reliably reflected in the requester's view). Role-based access filtering (SRS-5, SRS-16) maps naturally onto SQL WHERE clauses with user-context parameters, making Restricted-request visibility straightforward to implement and audit. The access-log requirement (SRS-17) is equally well-served by a relational table with foreign keys to the request and user records. User identity and role data (SRS-18, SRS-19, SRS-20) is equally structured — each user record holds a Google account identifier, display name, email, role, and registration timestamp — and fits naturally into the same relational store

without requiring a separate identity database. Given the modest data volume of a single lab environment and the development team size, a relational database keeps the technology stack simple to operate and query without the added operational complexity of a separate NoSQL engine.

simple to operate and query without the added operational complexity of a separate NoSQL engine.

Component	Data Storage Technology	Description
Request Database	RDBMS (e.g., PostgreSQL / MySQL)	Primary relational store with normalised tables for: requests (all standard fields and status), users/roles, industry-collaboration details (linked to requests), and request-history/closed records. Supports all SRS-1 through SRS-16 read/write operations with integrity constraints and transactional updates.
Access Log Store	RDBMS — dedicated log table	A structured log table recording (request_id, user_id, role, access_timestamp) for every read of a Restricted request (SRS-17). Co-located in the same RDBMS instance for simplicity; can be migrated to a dedicated audit database if volume grows in later iterations

11. Development Environment

The recommended development and deployment environment is a containerised environment using Docker / Docker Compose, rather than bare-metal or a full virtual machine per service.

Rationale

The system must be accessible to multiple user roles (Lab Members on personal laptops, Lab Manager on lab workstations, the Lab Manager on a dedicated machine) within the same lab network. A containerised environment packages the application, its dependencies, and the RDBMS together into a portable, reproducible unit, so the development team can move the system between development machines and the lab's production server without reconfiguring dependencies each time. Bare-metal deployment would tie the application to one specific machine configuration, making it fragile to reproduce if hardware changes or if a second server is added. Docker Compose orchestrates the application container and the database container together, keeping the RDBMS instance isolated from the host system and making it easy to reset or redeploy the environment quickly — valuable when there is no dedicated IT-support team. Role-based access and Restricted-request confidentiality (SRS-16) also benefit from a consistent, isolated environment where the authentication module behaves identically across development, testing, and production.

The application container requires Google OAuth 2.0 credentials (Client ID and Client Secret) to be supplied as environment variables (SRS-18), keeping sensitive credentials out of the codebase and consistent across development and production environments.

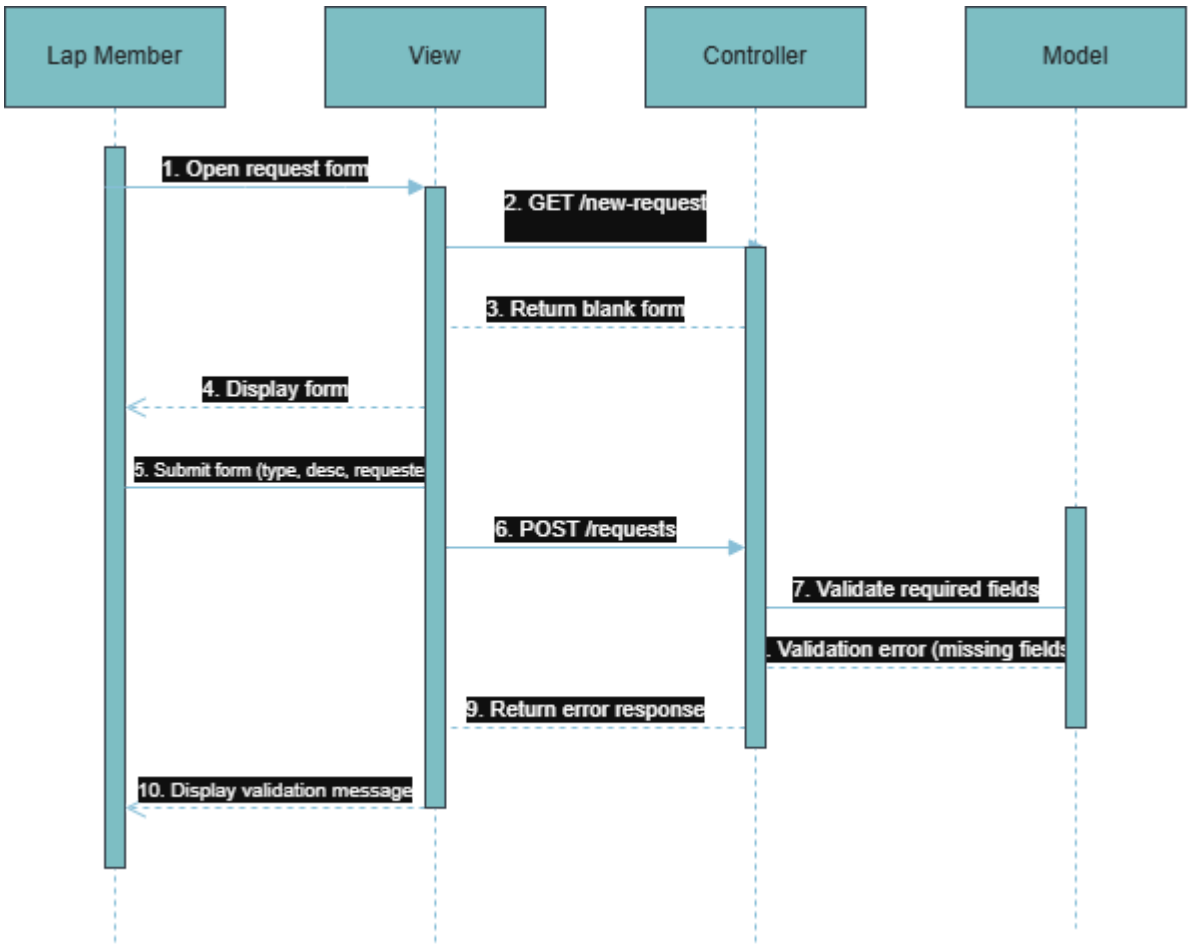
Component	Development Environment	Description
Application Server (Controller + Model)	Docker container	Runs the MVC application logic in an isolated, reproducible container image that can be deployed identically on a development laptop or the lab's production server.
Request Database (RDBMS)	Docker container (via Docker Compose)	Runs the relational database in its own container, orchestrated alongside the application container using Docker Compose, keeping data storage decoupled from the host machine's configuration.
Local Development Machine	Local Docker Desktop	Used by the developer team to build and test the system before

		deployment, mirroring the production container configuration to minimise environment inconsistencies.
Production Server (Lab Network)	Docker host (Linux VM or bare metal)	Hosts the Docker Compose stack in the lab network, accessible to all user roles via a web browser. TLS termination handles secure connections for Restricted-request confidentiality compliance

12. Sequence Diagrams for All SRS

The following sequence diagrams are derived from the chosen MVC modular architecture (Section 9) and cover all use cases traced in the SRS traceability matrix.

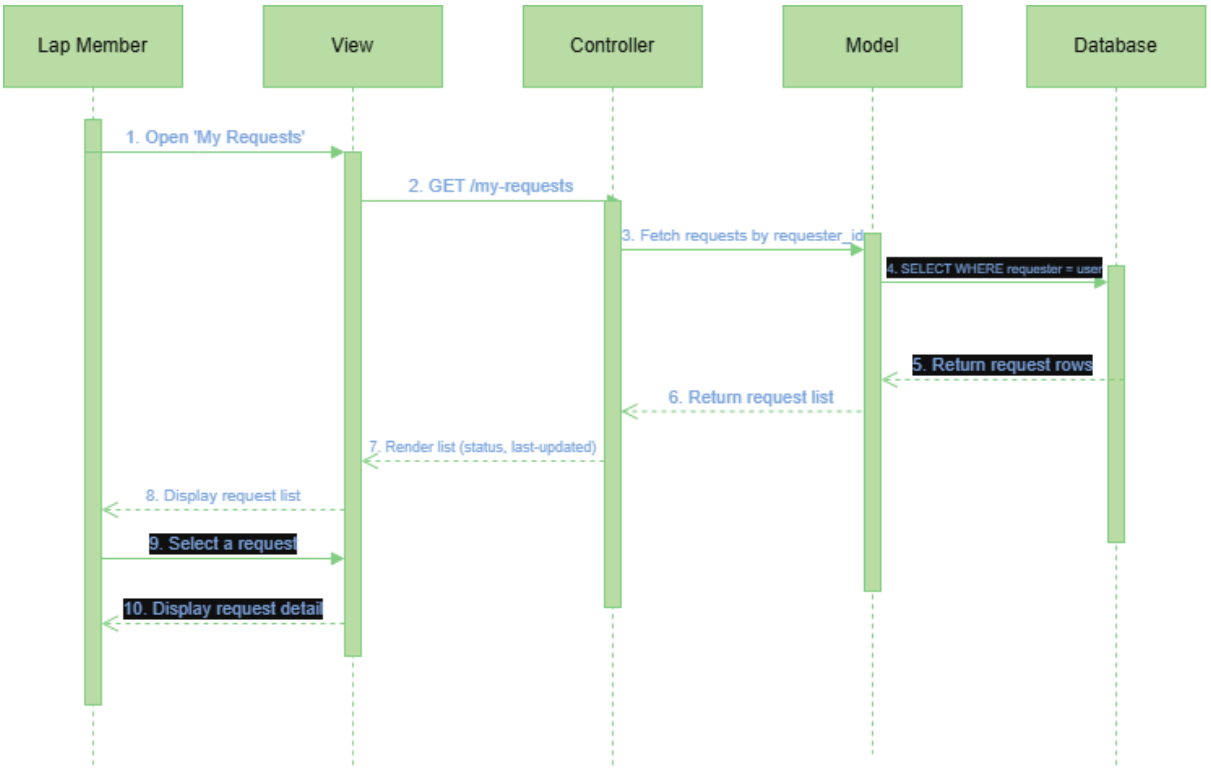
SQD-1: Lab Member Submits a Request
Traced requirements: SRS-1, SRS-2, SRS-3



On the success path (all required fields present), step 8 returns 'Valid'; the Controller calls the Model to assign status 'New' and record the timestamp (SRS-3), the Model persists the record, and the View shows a submission confirmation.

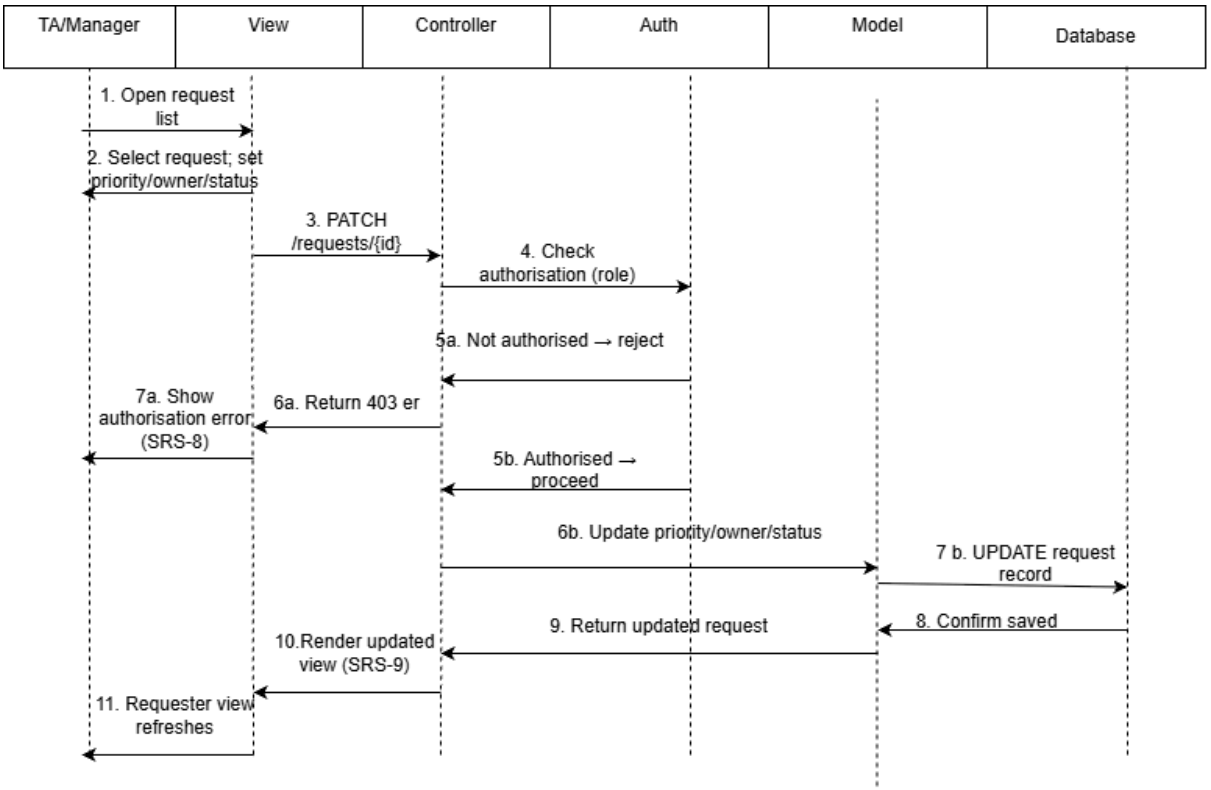
SQD-2: Lab Member Views Own Request Status

Traced requirements: SRS-4, SRS-5



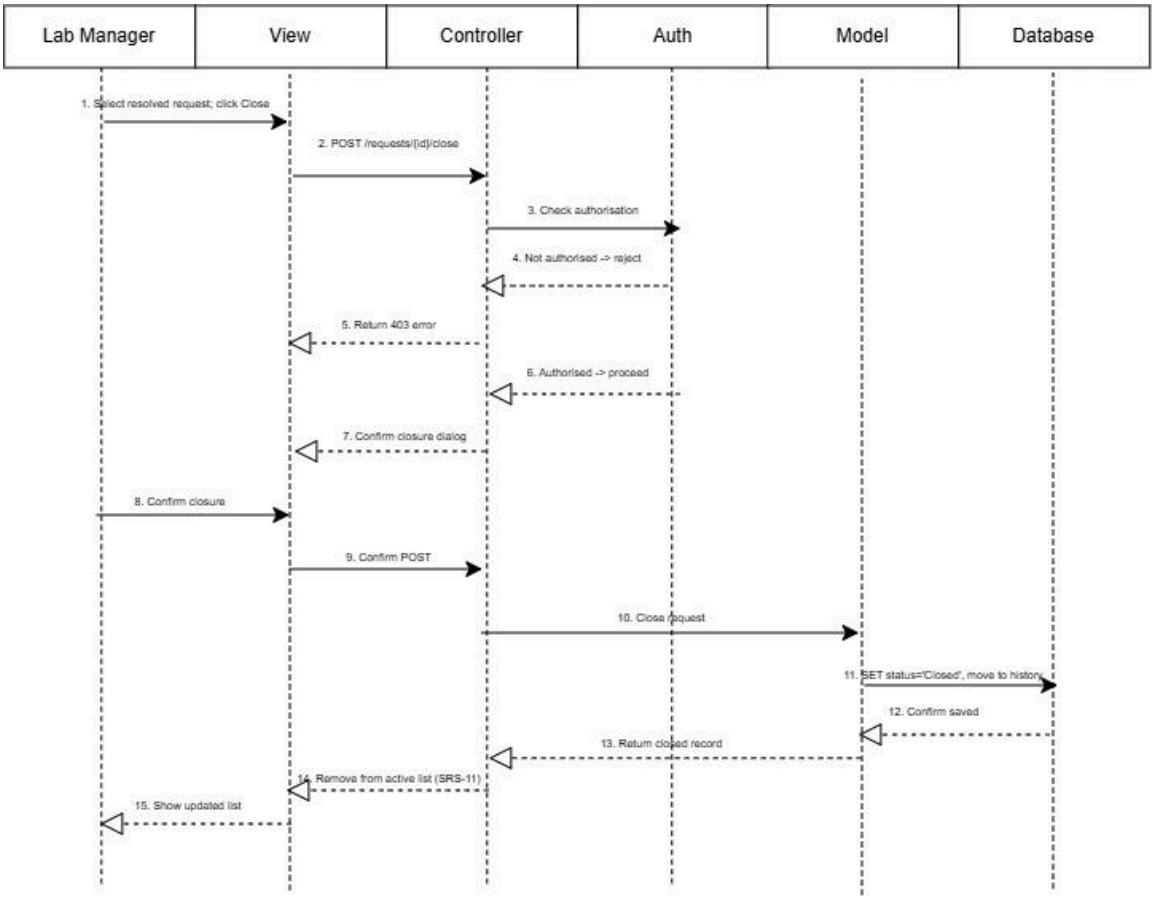
SQD-3: Lab Manager Triages and Updates Request Status

Traced requirements: SRS-6, SRS-7, SRS-8, SRS-9



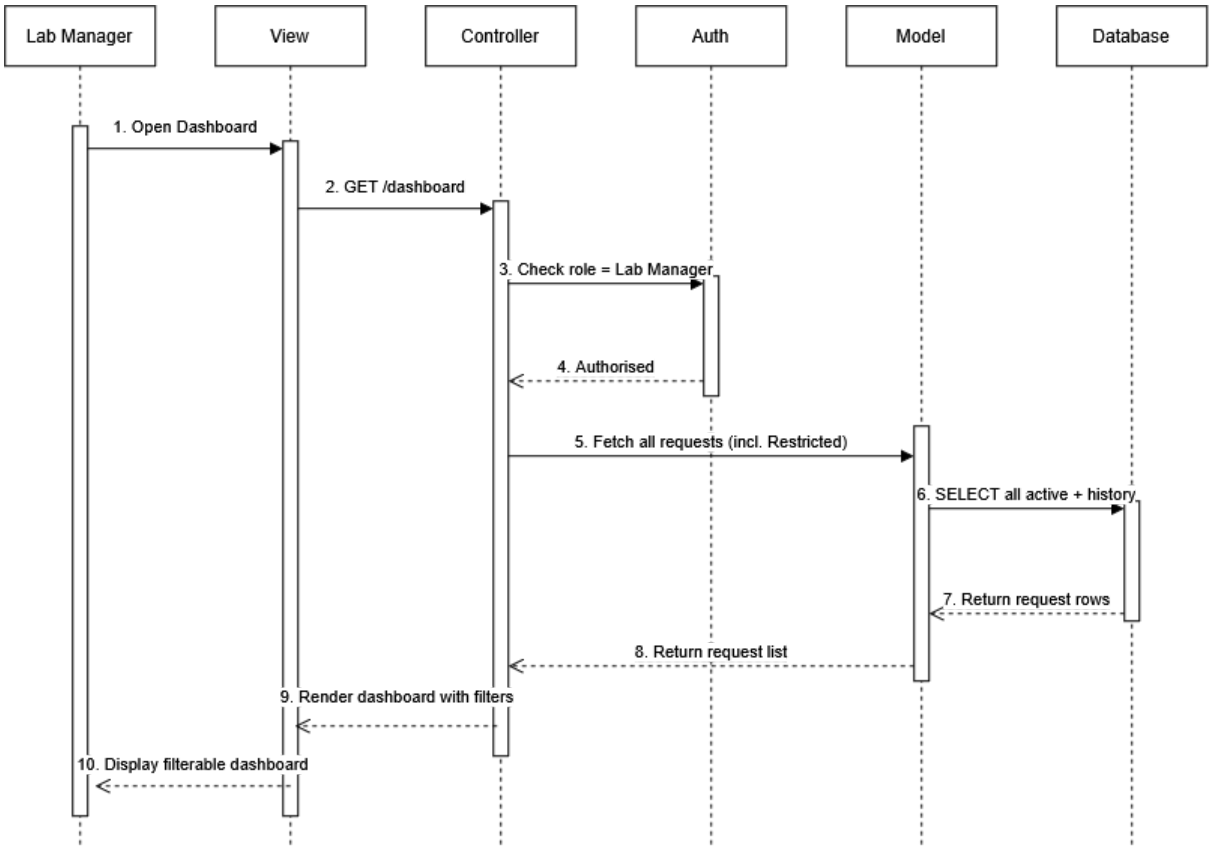
SQD-4: Lab Manager Closes a Request

Traced requirements: SRS-10, SRS-11



SQD-5: Lab Manager Views All Requests (Dashboard)

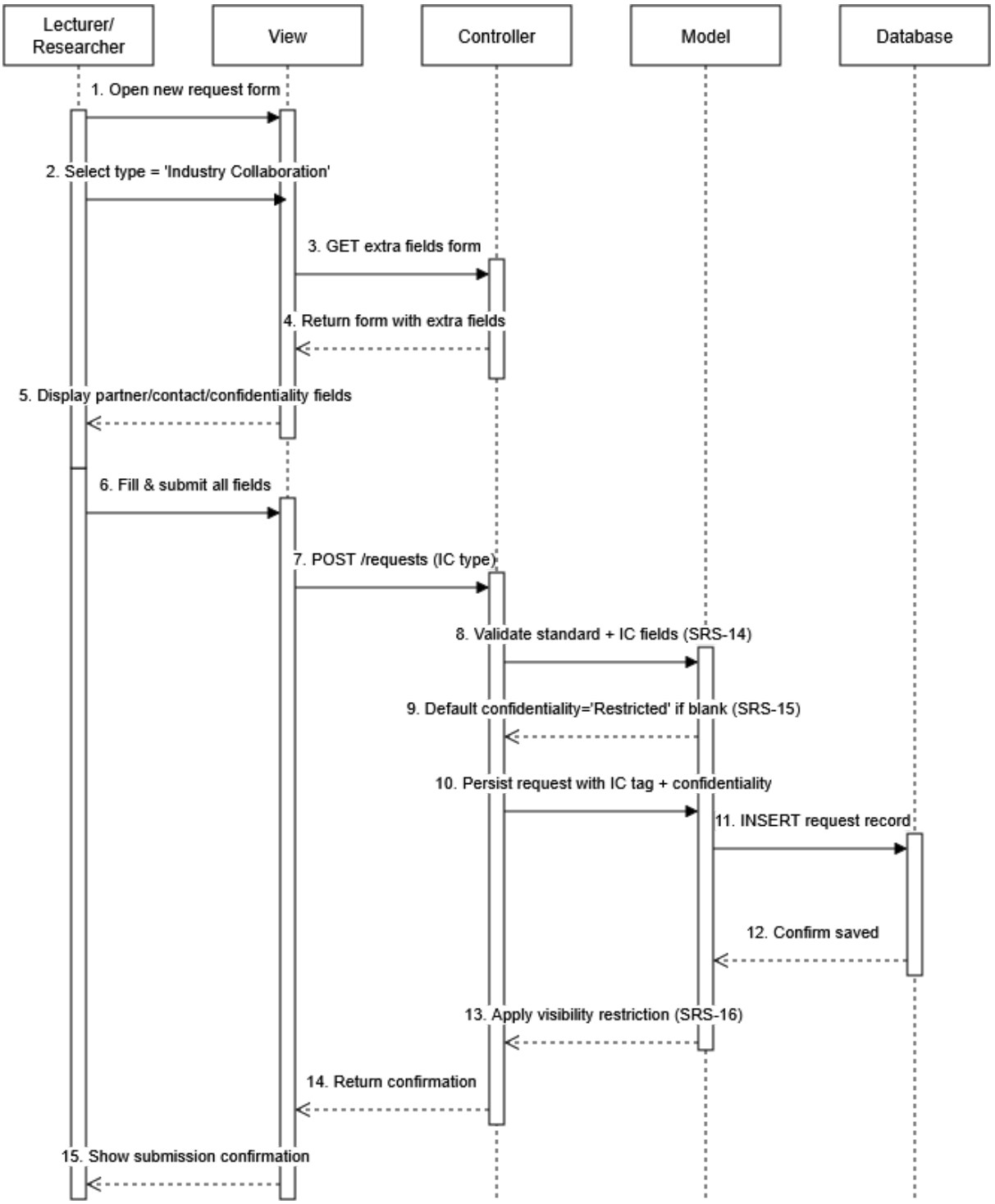
Traced requirements: SRS-12



The Lab Manager's dashboard includes Restricted requests (SRS-12 + SRS-16). Filters by status, type, and requester are applied client-side or via query parameters passed to the Controller.

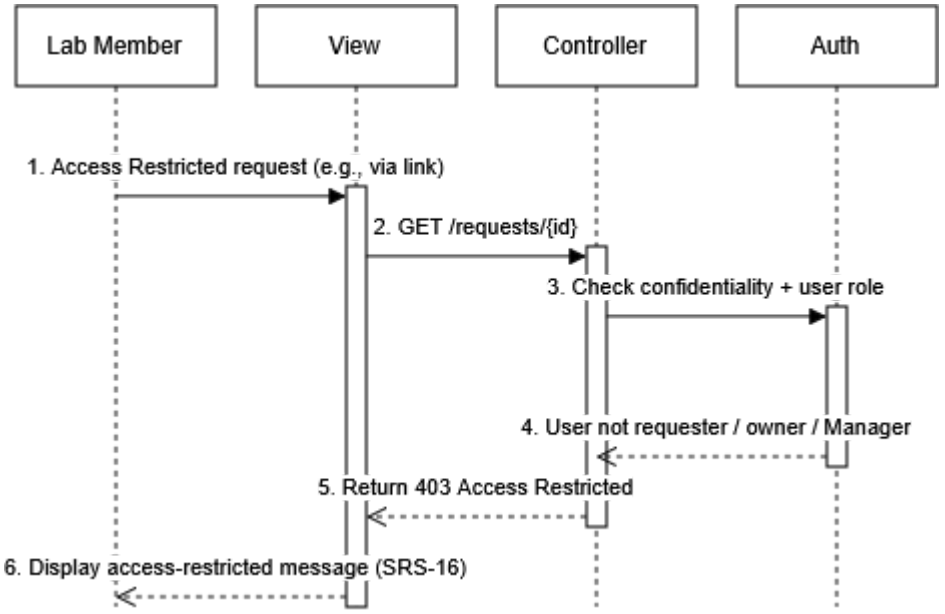
SQD-6: Lecturer / Researcher Submits Industry Collaboration Request (Success Path)

Traced requirements: SRS-14, SRS-15, SRS-16



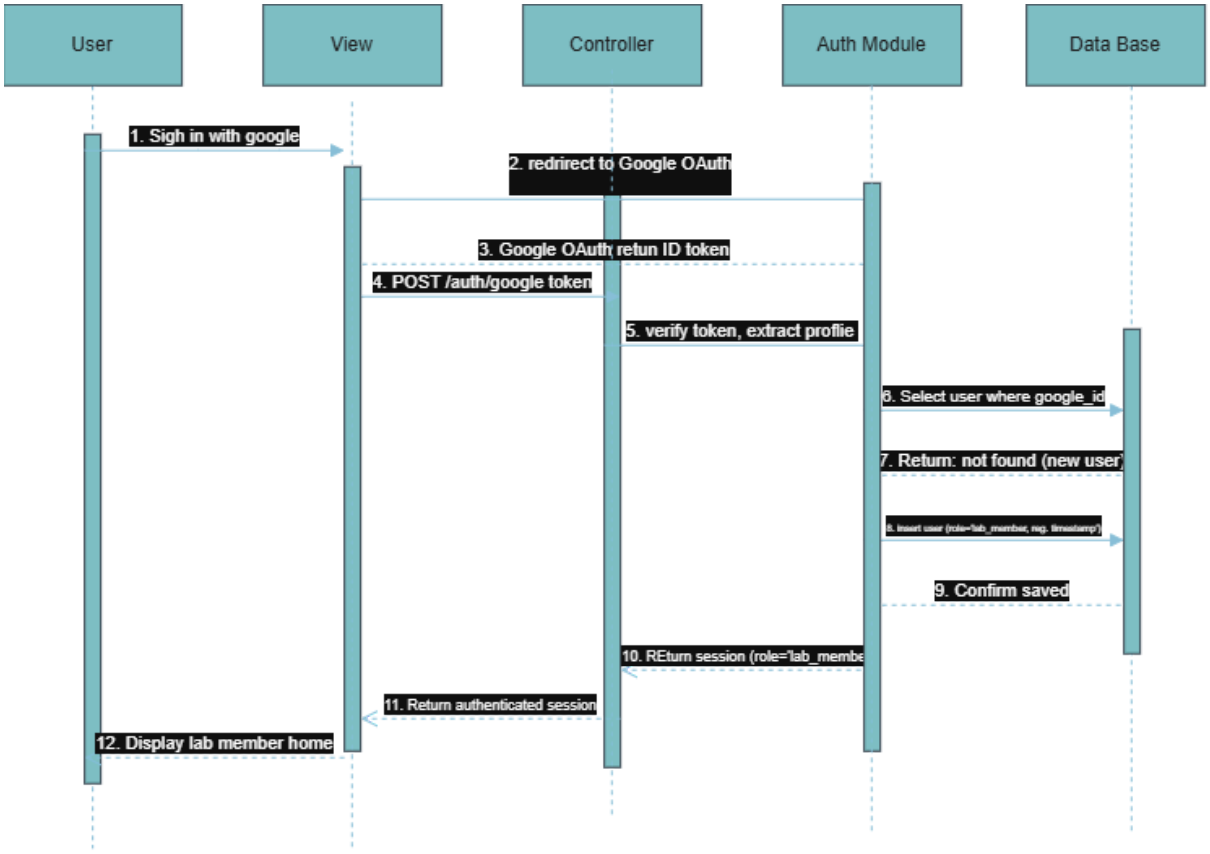
SQD-7: Restricted Request Access Control (Unauthorised Lab Member Attempt)

Traced requirements: SRS-16, SRS-17



On the authorised path (requester, assigned owner, or Lab Manager), step 4 returns 'Authorised'; the Controller proceeds to fetch the full request detail, the Access Log Module records (user_id, request_id, timestamp) for accountability (SRS-17), and the View renders the full Restricted request.

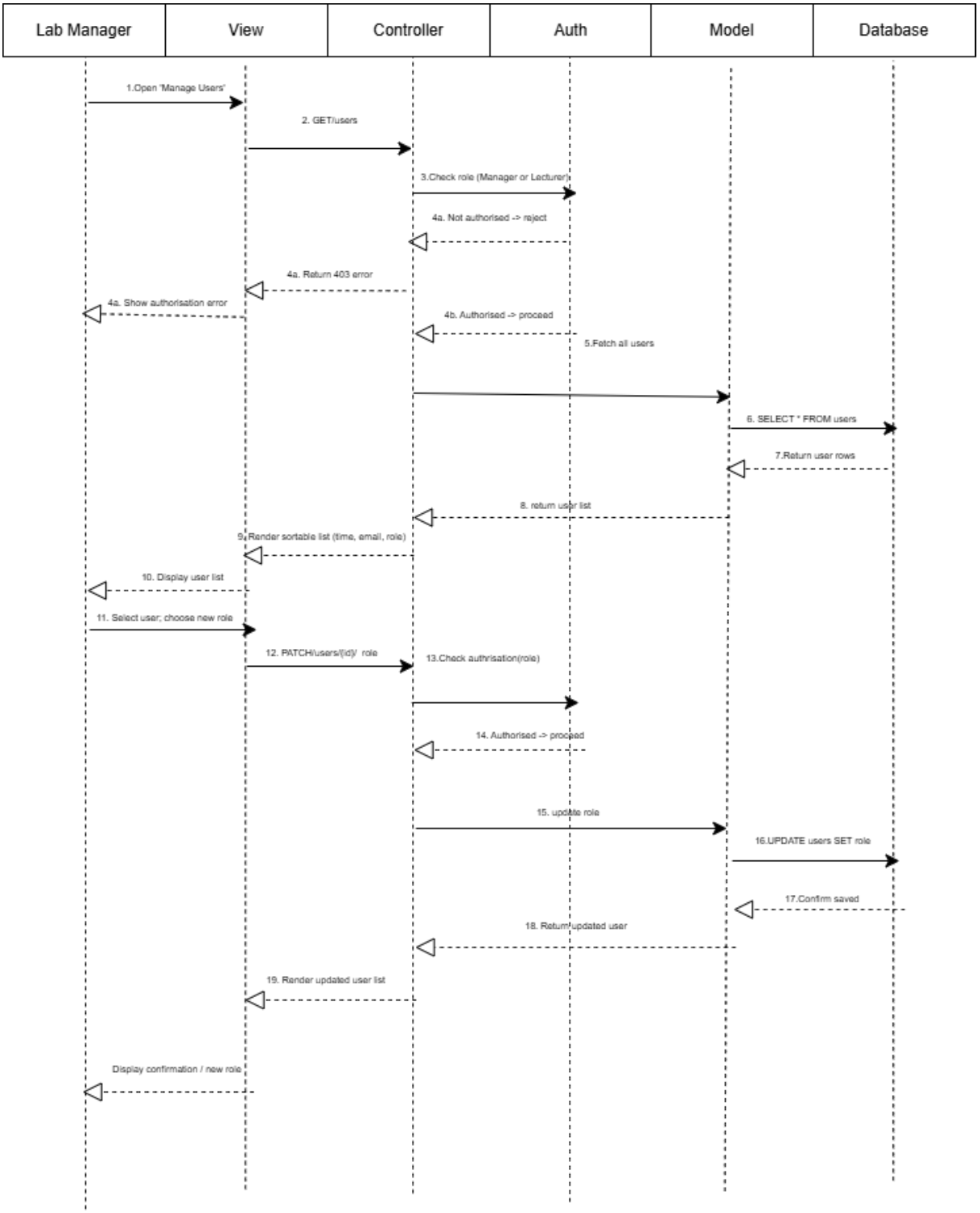
SQD-8: User Signs In via Google OAuth (New User Auto-Registration) Traced
requirements: SRS-18, SRS-19



The user authenticates through Google OAuth 2.0 (SRS-18). The Auth Module verifies the returned token and checks the Database for an existing account. On the path shown — no matching record — the Auth Module creates a new user record with role "lab_member" and records the registration timestamp (SRS-19) before returning an authenticated session. On the alternate path (returning user), step 7 returns the existing record instead, steps 8–9 are skipped, and the session is issued with the user's stored role.

SQD-9: Lab manager / Lecturer Changes a User's Role

Traced requirements: SRS-20



13. Traceability Matrix

Sub-Goal	Use Case	User Story	URS	Activity Diagram	SRS
SG-1	UC-1 Submit Request UC-2 View My Request	US-1 US-2	URS-1, URS-2	AD-1 AD-2	SRS-1, SRS-2, SRS-3 SRS-4, SRS-5
SG-2	UC-1 Submit Request	US-1	URS-1	AD-1 (Submit Request)	SRS-1, SRS-2, SRS-3
SG-3	UC-3 Triage/Assign Request UC-4 Update Request Status	US-3 US-4	URS-3 URS-4	AD-3	SRS-6, SRS-8 SRS-7 , SRS-8, SRS-9
SG-4	UC-3 Triage/Assign Request UC-4 Update Request Status	US-3 ,US-4	URS-3, URS-4	AD-3	SRS-8, SRS-9, SRS-13
SG-5	UC-5 Close Request UC-6 View All Requests (Dashboard)	US-5 US-6	URS-5 URS-6	AD-4	SRS-10, SRS-11 SRS-12
SG-6	linking to UC "Submit Industry Collaboration Request,"	US-7 US-8	URS-7 URS-8	AD-5	SRS-14, SRS-17

From SRS to Sequence Diagram

SRS	SQD
SRS-1	SQD-1
SRS-2	SQD-1
SRS-3	SQD-1
SRS-4	SQD-2
SRS-5	SQD-2
SRS-6	SQD-3
SRS-7	SQD-3
SRS-8	SQD-3
SRS-9	SQD-3
SRS-10	SQD-4
SRS-11	SQD-4
SRS-12	SQD-5
SRS-13	SQD-1,SQD-3
SRS-14	SQD-6
SRS-15	SQD-6
SRS-16	SQD-6,SQD-7
SRS-17	SQD-7
SRS-18	SQD-8
SRS-19	SQD-8
SRS-20	SQD-9